

■ 2.7.3 Using the Same Name for Different Things

In standard mathematical notation, one often ends up using the same symbol to mean several different things. In one context, for example, the symbol p might stand for a prime number; in another context, it might correspond to something very different, say momentum.

When you use *Mathematica*, you should always try to avoid choosing the same name for several different things. However, particularly when you use several packages at the same time, it is sometimes inevitable that the same name will be used in different places for different purposes.

Mathematica supports the notion of “contexts”, which allow it to distinguish different uses of a particular name. Instead of just referring to a symbol by a name like `p`, you can specify the context for the symbol explicitly. Thus for example, you can type `NumberTheory`p` or `Physics`p` to distinguish symbols with the name `p` from two different contexts. Of course, if a particular name appears only in one context, you do not have to give the name of the context explicitly.

Every symbol that you use in *Mathematica* is in fact associated with one context or another. The full specification of any symbol is therefore `context`name`. The ``` that appears here is called a *Mathematica* “context mark”.

As one example, all built-in *Mathematica* objects are in the context `System``. The full specification of the function `Integrate`, for example, is therefore `System`Integrate`.

Many external packages that you read in to *Mathematica* set up their own contexts. Usually the names of the contexts are based on the names of the packages. By having different contexts for different packages, one makes sure that objects from one package cannot get confused with objects from another package.

This is a symbol with name <code>x</code> in context <code>a</code> .	<pre>In[1]:= a`x Out[1]= a`x</pre>
<code>a`x</code> and <code>b`x</code> are different symbols. They both have <i>name</i> <code>x</code> , but they have different contexts.	<pre>In[2]:= a`x - b`x Out[2]= a`x - b`x</pre>
This gives the context of <code>a`x</code> .	<pre>In[3]:= Context[a`x] Out[3]= a`</pre>
The context for built-in functions is <code>System`</code> .	<pre>In[4]:= Context[Integrate] Out[4]= System`</pre>

<code>context`name</code>	the full name of a symbol, with the context given explicitly
<code>Context[symbol]</code>	the context for a particular symbol

Contexts for symbols.